Day 34: Deploying a Model Behind a Minimal FastAPI Endpoint

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 7

Day 33 saved a fitted pipeline to disk. Today that file finally gets used for what persistence exists to enable: answering live prediction requests over HTTP, served by FastAPI, without ever rerunning the training code. Two real gotchas surfaced and were fixed while building this — both are documented below with the actual errors they produced.

1. Why Put the Model Behind an API

Topic specification: Wrapping a persisted pipeline in a small web server process that loads the model once and answers prediction requests sent to it over HTTP, from any client on the network.

Explanation: A saved .joblib file is inert until something loads it. An API is the simplest way to turn that dormant file into something other systems can actually use — a mobile app, a website backend, another internal service — without any of those callers needing Python, scikit-learn, or the model file itself installed locally.

Real-world scenario: An insurance company's underwriting team saves a risk-scoring pipeline with joblib. A completely separate customer-facing web application, written in a different language entirely, sends applicant details to a FastAPI endpoint and gets a risk score back as JSON — the web app never needs to know Python or scikit-learn exist.

Implementation:

```
app = FastAPI(title="Titanic Survival Predictor")

@app.post("/predict")
def predict(passenger: PassengerInput):
    ...
    return PredictionOutput(survived=..., survival_probability=...)
```

Line-by-line explanation

• `FastAPI()` — creates the application object; every route (URL + HTTP method combination) gets registered onto it with a decorator.

• `@app.post("/predict")` — declares that sending an HTTP POST request to /predict routes to this function; FastAPI handles all the HTTP-level plumbing.

Why choose this? An API decouples 'who trained the model' from 'who uses the model' completely — any client capable of sending an HTTP request can get predictions, regardless of what language or environment it runs in.

Why NOT this (tradeoffs)? Running a persistent server process is more operational overhead than a one-off script — it needs to be kept running, monitored, and restarted on failure, which a simple batch script never has to worry about.

2. Pydantic Schemas Validate Every Request

Topic specification: A Pydantic BaseModel (PassengerInput) declares the exact shape, types, and constraints an incoming request body must satisfy — FastAPI validates every request against it automatically, before the handler function ever runs.

Explanation: Without a schema, a handler would need to manually check that every field exists, has the right type, and falls in a sane range — tedious, error-prone code that's easy to forget a case for. Pydantic does all of that declaratively from type hints and Field constraints.

Real-world scenario: A frontend developer building the form that calls this API can read the PassengerInput schema (FastAPI auto-generates interactive docs from it at /docs) to know exactly which fields are required, which are optional, and what range Pclass must fall in — without ever reading the Python implementation.

Implementation:

```
class PassengerInput(BaseModel):
    Pclass: int = Field(..., ge=1, le=3)
    Sex: str
    Age: Optional[float] = Field(None, ge=0, le=110)
    SibSp: int = Field(..., ge=0)
    Parch: int = Field(..., ge=0)
    Fare: float = Field(..., ge=0)
    Embarked: Optional[str] = None
```

Actual output:

```
POST /predict with invalid input (Pclass=5, out of the 1-3 range)
422 {'detail': [{'type': 'less_than_equal', 'loc': ['body', 'Pclass'],
     'msg': 'Input should be less than or equal to 3', 'input': 5, 'ctx': {'le': 3}}]}
```

Actual output from this lesson's run — rejected automatically, no handler code involved.

Line-by-line explanation

• `Field(..., ge=1, le=3)` — the `...` marks the field as required with no default; `ge`/`le` enforce the numeric range directly in the schema.

• `Optional[float] = None` — Age can be omitted or explicitly null in the request JSON; everywhere else in this schema, a missing field is rejected instead.

• Verified in this lesson: sending Pclass=5 never reaches the predict() function body at all — FastAPI returns HTTP 422 with a precise error before the handler runs.

Why choose this? Declarative validation catches malformed requests at the boundary, with a precise machine- and human-readable error message, and keeps the handler function itself free of defensive boilerplate.

Why NOT this (tradeoffs)? Pydantic validation only catches structural problems (wrong type, out of range) — it cannot catch semantically wrong-but-well-formed input, like a Fare of 0.01 for a first-class ticket, which still requires separate business-logic checks if that matters.

3. Loading the Pipeline Once, at Startup

Topic specification: A lifespan context manager runs setup code once, before the app accepts any traffic, and teardown code once, when the app shuts down — the model is loaded exactly once per process lifetime, not once per request.

Explanation: Loading a joblib file from disk takes real time and I/O. Doing that inside the /predict handler would mean every single request pays that cost again, which is both wasteful and unnecessary since the loaded object never changes between requests.

Real-world scenario: A high-traffic API serving thousands of requests per minute loads its model once at process startup and keeps it in memory for the process's entire lifetime — reloading per request at that volume would make the service unusably slow.

Implementation:

```
model_state = {"pipeline": None}

@asynccontextmanager
async def lifespan(app: FastAPI):
    model_state["pipeline"] = joblib.load(MODEL_PATH)
    print(f"Model loaded from {MODEL_PATH}")
    yield
    model_state["pipeline"] = None

app = FastAPI(lifespan=lifespan)
```

Actual output:

```
Model loaded from /tmp/day34_assets/titanic_pipeline.joblib
```

Actual output from this lesson's run — printed once, at startup, not per request.

Line-by-line explanation

• `model_state = {"pipeline": None}` — a plain dict used as shared, mutable storage; request handlers read from it without needing a global statement.

• `yield` — everything before this line runs at startup; everything after runs at shutdown. The app serves requests while execution is paused at the yield.

• `lifespan=lifespan` — this is the current recommended pattern in FastAPI, replacing the older {C}@app.on_event("startup"){E} decorator, which is now deprecated.

Why choose this? Loading once at startup means every request only pays the (fast) cost of a dictionary lookup and a prediction, not a disk read — the difference between an API that scales and one that doesn't.

Why NOT this (tradeoffs)? If the underlying model file is updated on disk, this app won't see the new version until it's restarted — a genuine limitation lifespan-based loading doesn't solve, requiring a separate reload mechanism if hot-swapping models matters.

4. A Real Gotcha: Unpickling Needs the Original Function

Topic specification: Day 33's pipeline includes FunctionTransformer(add_family_size) — joblib/pickle store a reference to that function (its module and name), not its actual code, so loading the file elsewhere requires that exact function to be importable from the exact module it was pickled under.

Explanation: Pickle's handling of functions and classes works by reference, the same way it handles objects. When unpickling reconstructs the pipeline, it needs to resolve day33_model_persistence.add_family_size by actually importing that name from that module — if the new script never imports it, Python has no way to find it.

Real-world scenario: A backend engineer, new to the project, copies just the saved .joblib file into a fresh microservice repository without the original training script — loading it fails immediately with a cryptic AttributeError, until someone realizes the custom function needs to travel with the model file, not just the file itself.

Implementation:

```
# Reproduced directly in this lesson's sandbox, without the import:
#   AttributeError: Can't get attribute 'add_family_size' on
#   <module '__main__' from 'day34_fastapi_deployment.py'>

# Fixed with this import — never called directly, but makes the name
# resolvable when joblib.load() reconstructs the pipeline:
from day33_model_persistence import add_family_size  # noqa: F401
```

Actual output:

```
Traceback (most recent call last):
  ...
  File "pickle.py", line 1582, in find_class
    return _getattribute(sys.modules[module], name)[0]
AttributeError: Can't get attribute 'add_family_size' on <module '__main__' ...>
```

Actual traceback reproduced in this lesson's sandbox before the fix was applied.

Line-by-line explanation

- The traceback's own file path confirms where the resolution fails: pickle looked for add_family_size inside the module that is loading the file (day34's own module), not the module that originally defined it.
- `# noqa: F401` — a linting comment; the import looks 'unused' to a static checker since it's never called by name, even though it's essential for joblib.load() to succeed.

Why choose this? This failure mode is loud and immediate — an AttributeError, not a silently wrong prediction — so it's caught the first time the app is started rather than surfacing subtly in production.

Why NOT this (tradeoffs)? It creates a real coupling between the serving script and the training script's module structure — renaming or deleting add_family_size in day33_model_persistence.py, or moving it, would break Day 34's app even though Day 34's own code never changed.

5. The Same None-vs-NaN Bug, In a New Place

Topic specification: Pydantic deserializes a JSON null into Python's None for Optional fields. SimpleImputer's default missing_values=np.nan check — already shown in Day 33 to not recognize plain None — resurfaces here, now triggered by real API requests instead of hand-built test data.

Explanation: A client sending {"Embarked": null} is completely normal, valid JSON for a missing optional field. FastAPI/Pydantic correctly turns that into Python's None. But None is not the same object as np.nan, and SimpleImputer's default check does not treat them as equivalent on object-dtype columns — verified directly in this lesson.

Real-world scenario: A production API silently mis-handles every request where a client omits an optional field as JSON null, quietly degrading predictions instead of failing loudly — exactly the kind of bug that's easy to miss in testing (which often uses hand-built, non-null test data) and only shows up once real traffic sends genuinely missing fields.

Implementation:

```
row = pd.DataFrame([passenger.model_dump()])
# Optional fields deserialize JSON null -> Python None, which
# SimpleImputer's default missing_values=np.nan does not catch.
# .where() converts every None cell to a real np.nan before predicting.
row = row.where(row.notna(), np.nan)
prediction = int(pipeline.predict(row)[0])
```

Actual output:

scenario	Embarked value	warning raised?	probability
before the fix	None (unconverted)	yes — unknown categories	0.6285
after the fix	np.nan (converted)	no	0.5291

Actual output from this lesson's run, same request, before and after the fix.

Line-by-line explanation

• `passenger.model_dump()` — converts the validated Pydantic object back into a plain dict; any Optional field the client omitted or sent as null is present here as Python None.

• `row.where(row.notna(), np.nan)` — `row.notna()` treats None as missing (pandas's own check is more lenient than sklearn's), producing a mask that replaces every missing cell with an actual np.nan float, not the original None object.

• Verified in this lesson: the exact same request produced a UserWarning and a different, less trustworthy probability before this fix, and a clean run with no warning after it.

Why choose this? This fix is one line, and it converts a class of bug that could otherwise silently degrade every request with a genuinely missing optional field — cheap insurance against a recurring, well-understood failure mode.

Why NOT this (tradeoffs)? The underlying mismatch (None vs np.nan) is a real inconsistency between pandas's own missing-value handling and scikit-learn's default check — it's easy to reintroduce this bug anywhere new code constructs a DataFrame from raw Python dicts or JSON without this same conversion step.

6. Testing With TestClient, Then Running It For Real

Topic specification: FastAPI's TestClient exercises the whole application in-process — including the lifespan startup/shutdown events — without opening a real network socket or needing a separate terminal running a server.

Explanation: TestClient wraps the ASGI application directly, simulating HTTP requests against it in-memory. Used as a context manager (with TestClient(app) as client:), it triggers the lifespan startup before the first request and shutdown after the block exits, exactly mirroring what a real deployment does.

Real-world scenario: A CI pipeline runs this exact verification block on every code change, catching a broken endpoint or a regressed prediction before the code is ever deployed to a real server — no actual network calls, no test infrastructure beyond Python itself, and it runs in milliseconds.

Implementation:

```
with TestClient(app) as client:
    resp = client.get("/health")
    resp = client.post("/predict", json={
        "Pclass": 1, "Sex": "female", "Age": 29, "SibSp": 0,
        "Parch": 0, "Fare": 100.0, "Embarked": "S",
    })

# For real traffic, the same `app` object is served with:
#   uvicorn day34_fastapi_deployment:app --reload
```

Actual output:

Pclass	Sex	Age	Fare	Embarked	survived	probability
1	female	29	100.00	S	True	0.9711
3	male	22	7.25	S	False	0.0263
2	female	None	26.00	None	True	0.5291

Actual output from this lesson's run — 3 requests through the real /predict endpoint.

Line-by-line explanation

• `with TestClient(app) as client:` — the `with` is what triggers the lifespan startup event; using TestClient(app) without it can leave model_state["pipeline"] unset.

• GET /health returned 200 with model_loaded: true, confirming the lifespan startup ran successfully before any prediction request was sent.

• The malformed request (Pclass=5) from Topic 2 and the None-handling fix from Topic 5 were both verified inside this same TestClient session, alongside the three valid predictions shown above.

Why choose this? In-process testing is fast, deterministic, and requires no separate server process — ideal for automated tests and for verifying an app works before ever deploying it.

Why NOT this (tradeoffs)? TestClient does not exercise real network behavior (actual sockets, real HTTP timeouts, load balancing) — a final check against a real running uvicorn server is still worthwhile before trusting a deployment completely.

Interview Preparation — Technical Questions

Q: Why does the model get loaded inside a lifespan function instead of at the top of the module or inside the /predict handler?

A: Loading inside the handler would repeat the (relatively slow) disk read on every single request. Loading at the very top of the module happens at import time, which can complicate testing and doesn't cleanly support teardown logic. lifespan runs setup exactly once when the app actually starts serving traffic, and teardown exactly once when it stops, which is the correct lifecycle for a resource like a loaded model.

Q: What specific error does removing the add_family_size import produce, and why?

A: AttributeError: Can't get attribute 'add_family_size' on — because joblib/pickle stored only a reference (module path + function name) to that function when the pipeline was saved, and unpickling needs to resolve that exact reference by importing it from the exact module it was defined in.

Q: Why does Pydantic's Optional[float] = None field combined with SimpleImputer produce a silent warning instead of a hard error?

A: SimpleImputer's default missing_values=np.nan check does not recognize plain Python None on object-dtype columns, so the value passes through unimputed to the OneHotEncoder, which treats it as an unrecognized category and emits a UserWarning rather than raising — verified directly in this lesson by comparing the same request before and after the fix.

Q: What does Field(..., ge=1, le=3) do differently from a plain int type hint?

A: The plain type hint only enforces that the value is an integer. ge (greater-than-or-equal) and le (less-than-or-equal) add numeric range constraints that Pydantic checks automatically during validation, rejecting out-of-range values with a structured error before the handler function ever executes — verified in this lesson with a Pclass=5 request returning 422.

Q: Why does using TestClient as a context manager (with TestClient(app) as client:) matter specifically for this app?

A: This app's model is loaded inside the lifespan function, which only fires on the ASGI lifespan startup event. TestClient only sends that event when used as a context manager; instantiating it without with can leave model_state['pipeline'] as None, causing every prediction request to fail with the 503 raised in the handler for that case.

Q: Why is model_state implemented as a dict instead of a plain module-level variable?

A: A plain variable reassigned inside the lifespan function would require a global statement and can behave unexpectedly with how Python resolves closures and module reloading in some testing setups. A dict's contents can be mutated in place from any function without a global declaration, which is a common, simple pattern for small pieces of shared app state.

Q: What HTTP status code does FastAPI return for a request that fails Pydantic validation, and what does the response body contain?

A: HTTP 422 Unprocessable Entity, with a JSON body listing every failed field: its location, the validation rule that failed, the offending input value, and a human-readable message — verified in this

lesson's actual output for the Pclass=5 case.

Q: Why does uvicorn day34_fastapi_deployment:app --reload use a colon between the filename and the variable name?

A: The part before the colon is the importable module path (the file without .py); the part after is the name of the FastAPI instance variable defined in that module. uvicorn imports the module and looks up that exact variable name to get the ASGI application object it should serve.

Q: Why does the /health endpoint check model_state['pipeline'] is not None instead of just returning a fixed 'ok' response?

A: A fixed response would report healthy even if the model failed to load during startup — a genuinely broken deployment reporting itself as fine. Checking the actual loaded state makes /health a meaningful signal that monitoring tools or load balancers can rely on to detect a real problem.

Interview Preparation — Theoretical Questions

Q: Why does deploying a model behind an API represent a fundamentally different kind of engineering problem than training one?

A: Training is a bounded, offline computation with a clear start and end. Serving is a long-running, always-on process that must handle concurrent requests, malformed input, partial failures, and continuous uptime — closer to general backend engineering than to data science, which is why deployment often involves a different skill set and different failure modes than the modeling work that precedes it.

Q: Why does the add_family_size unpickling failure illustrate a broader principle about what 'the model' actually consists of?

A: A trained pipeline is not a self-contained artifact — it has dependencies on the exact code (classes, functions, library versions) that existed when it was fitted. Treating the .joblib file alone as 'the model' understates what actually needs to be shipped and versioned together for the model to be reproducibly loadable anywhere.

Q: Why might input validation at the API boundary (Pydantic) be considered a form of contract between a model's producers and its consumers?

A: The schema documents, in a machine-checkable way, exactly what the model expects — its callers don't need to read the model's training code to know the valid input shape. This mirrors how a function's type signature acts as a contract in typed programming languages: callers can rely on it without inspecting the implementation.

Q: Why does the same None-vs-NaN bug recurring in a new context (Day 33's test data, Day 34's API requests) suggest it should be fixed at a more fundamental layer than either individual location?

A: A bug that resurfaces in every new context where raw data enters the pipeline suggests the fix belongs in the pipeline itself — for instance, a custom preprocessing step at the very start of the pipeline that normalizes None to np.nan unconditionally — rather than being re-patched at every call site that happens to construct a DataFrame from raw Python values.

Q: Why is loading the model once per process (not once per request) a specific instance of a much more general performance principle?

A: This is an instance of hoisting invariant work out of a hot path — computing something once outside a loop or repeated operation, rather than recomputing it identically every time. The same principle applies broadly: cache anything expensive and unchanging outside the code path that runs per unit of work, whether that's a loaded model, a compiled regex, or a database connection pool.

Q: Why does a health check endpoint matter more once a model is behind a persistent server process than it did as a one-off script?

A: A one-off script either runs successfully or crashes with a visible error — there's no ambiguity. A long-running server can enter a broken state silently (a failed model load, an exhausted resource) while still accepting connections, so external systems need an explicit, checkable signal of health rather than inferring it from the process simply being alive.

Q: Why does the lifespan pattern's clean separation of startup and shutdown logic matter more in a production deployment than it does in this lesson's TestClient verification?

A: In production, shutdown logic matters for graceful termination — releasing resources, finishing in-flight requests, avoiding corrupted state — during deployments, restarts, or scaling events that happen constantly at scale. This lesson's TestClient run experiences shutdown exactly once and briefly, which demonstrates the mechanism works but doesn't exercise the operational pressure that makes it matter in the first place.

Q: Why is 'the tests pass in TestClient' not sufficient evidence that a deployment is fully production-ready?

A: TestClient runs entirely in-process, sharing the same Python interpreter, memory space, and synchronous call stack as the test code — it cannot reveal issues specific to real network behavior (concurrency under load, connection handling, actual serialization over a real socket) or to the deployment environment itself (missing dependencies, different file paths, insufficient permissions) that only manifest once the app is actually served.

Key Takeaway and What's Next

Key takeaway: A persisted model becomes genuinely useful only once it's served behind an interface other systems can call. FastAPI plus Pydantic turns Day 33's saved pipeline into a validated, documented endpoint in a small amount of code — but two real gotchas (unpicklable custom functions, and None slipping past the imputer) show that persistence's edge cases follow the model wherever it's loaded next, not just where it was first saved.

What typically follows a minimal deployment

• `Model monitoring` — tracking the live distribution of incoming requests and predictions over time, to catch data drift before it silently degrades accuracy.

• `Containerization` — packaging the app, its dependencies, and the model file together (commonly with Docker) so the deployment environment is reproducible wherever it runs.

• `Week 5 review, then deep learning foundations` — Week 6 begins a new phase of the roadmap, with neural network fundamentals replacing XGBoost as the model architecture being trained, persisted, and eventually served the same way.